

PHYS 2550 Homework 1

Don van Beusichem

September 19, 2026

[GitHub](#)

1 Data and exploratory analysis

The given data is a UCI Real Estate Valuation, and contains 414 property transactions from Sindian District, New Taipei City, Taiwan [1]. I converted the supplied XLSX file to CSV directly in the Python notebook. `pandas.read_excel()` loaded the spreadsheet and `DataFrame.to_csv()` saved it. The CSV contains eight numeric columns and no missing values.

Table 1: Variables, units, and observed ranges. The row number is an identifier, not a feature.

Variable	Meaning and unit	Range
No	Row identifier; unitless	1 to 414
Transaction date	Decimal year	2012.667 to 2013.583
House age	Years	0 to 43.8
Distance to MRT	Metres	23.383 to 6488.021
Convenience stores	Count within walking distance	0 to 10
Latitude	Degrees	24.93207 to 25.01459
Longitude	Degrees	121.47353 to 121.56627
Price per unit area	10,000 New Taiwan dollars per ping	7.6 to 117.5

The target distribution in Fig. 1 is slightly right-skewed. Most prices are between about 20 and 60, however there is one observation near 118 which is rather far from the rest. House age forms several clusters. MRT distance is also strongly right-skewed: most houses are relatively close to a station, but there a few are 4–6.5 km away. The scales are also notably different, house age reaches about 44, whereas MRT distance reaches about 6488, which becomes important for our gradient descent in the future parts.

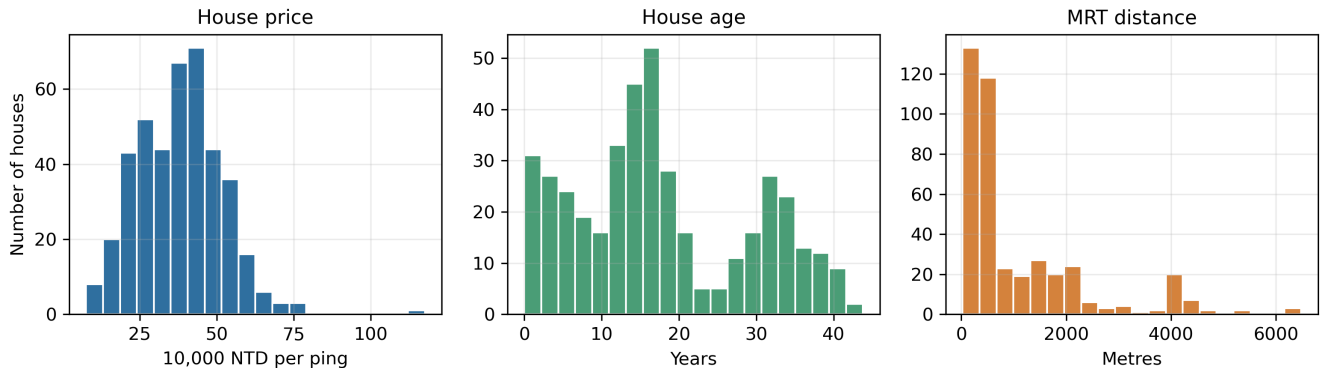


Figure 1: Distributions of the target, house age, and distance to the nearest MRT station.

2 One-feature linear regression

Using house age x_i as the only feature, the prediction for house i is

$$\hat{y}_i = w_0 + w_1 x_i. \quad (1)$$

For N houses, I minimized the mean squared error (MSE)

$$L(w_0, w_1) = \frac{1}{N} \sum_{i=1}^N (w_0 + w_1 x_i - y_i)^2. \quad (2)$$

Applying the chain rule gives

$$\frac{\partial L}{\partial w_0} = \frac{1}{N} \sum_i 2(w_0 + w_1 x_i - y_i) \frac{\partial(w_0 + w_1 x_i - y_i)}{\partial w_0} = \frac{2}{N} \sum_i (w_0 + w_1 x_i - y_i), \quad (3)$$

$$\frac{\partial L}{\partial w_1} = \frac{1}{N} \sum_i 2(w_0 + w_1 x_i - y_i) \frac{\partial(w_0 + w_1 x_i - y_i)}{\partial w_1} = \frac{2}{N} \sum_i x_i (w_0 + w_1 x_i - y_i). \quad (4)$$

Starting from $w_0 = w_1 = 0$, I updated each parameter using $w_j \leftarrow w_j - \eta \partial L / \partial w_j$. I chose $\eta = 0.001$ because it produced a smooth, stable decrease in loss, and used 20,000 epochs so the loss curve had ample time to reach a plateau.

The final values were

$$w_0 = 42.4343, \quad w_1 = -0.2515, \quad \text{MSE} = 176.5005. \quad (5)$$

Figure 2 confirms convergence but also shows weak predictive performance. Predictions occupy a narrow range of roughly 32–42 even though true prices range from 7.6 to 117.5. Clearly, the house age alone is not a strong predictor: inexpensive houses tend to be overpredicted and expensive houses underpredicted.

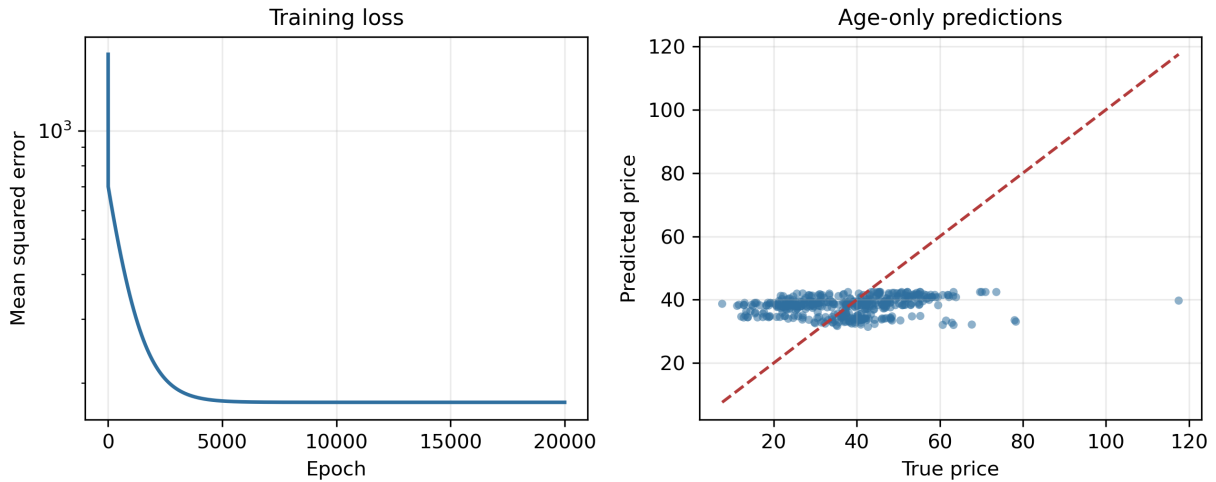


Figure 2: Left: MSE decreases and reaches a plateau. Right: age-only predictions compared with the perfect-prediction line.

3 Multiple features and normalization

I next used house age and distance to the nearest MRT station. With a constant 1 appended to every feature vector, the model is

$$\hat{\mathbf{y}} = X\mathbf{w}, \quad L(\mathbf{w}) = \frac{1}{N} (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}). \quad (6)$$

For weight w_j ,

$$\frac{\partial L}{\partial w_j} = \frac{2}{N} \sum_i X_{ij}(\hat{y}_i - y_i), \quad (7)$$

so stacking all partial derivatives gives

$$\nabla_{\mathbf{w}} L = \frac{2}{N} X^T (X\mathbf{w} - \mathbf{y}), \quad \mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} L. \quad (8)$$

Using the raw features with the same learning rate caused immediate divergence (Fig. 3). MRT distances of several thousand create much larger gradient terms than house ages of a few tens. The updates therefore overshoot the minimum, and each subsequent update grows larger.

I standardized each non-constant feature using

$$x_{\text{norm}} = \frac{x - \mu_x}{\sigma_x}. \quad (9)$$

This makes both feature means zero and standard deviations one. With the same $\eta = 0.001$ and 20,000-epoch limit, gradient descent converged smoothly to

$$\mathbf{w} = [37.9802 \quad -2.6288 \quad -9.0871]^T, \quad \text{MSE} = 93.9798. \quad (10)$$

Because the features are normalized, the slopes describe changes per standard deviation. Holding the other feature fixed, a one-standard-deviation increase in house age changes predicted price by -2.63 , whereas the same increase in MRT distance changes it by -9.09 .

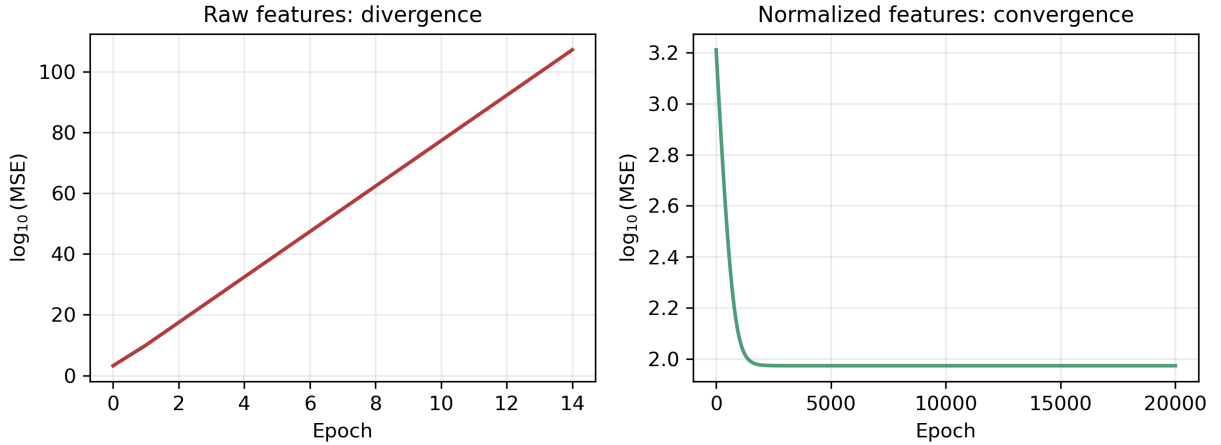


Figure 3: The raw-feature loss grows explosively, while normalization gives stable convergence with the same learning rate.

4 Model comparison

Adding MRT distance reduced MSE from 176.50 to 93.98, a 46.8% reduction. The multi-feature predictions cover a wider range and generally lie closer to the perfect-prediction line (Fig. 4). However, neither model captures the most expensive observation, and the linear model still compresses predictions toward the center of the target distribution.

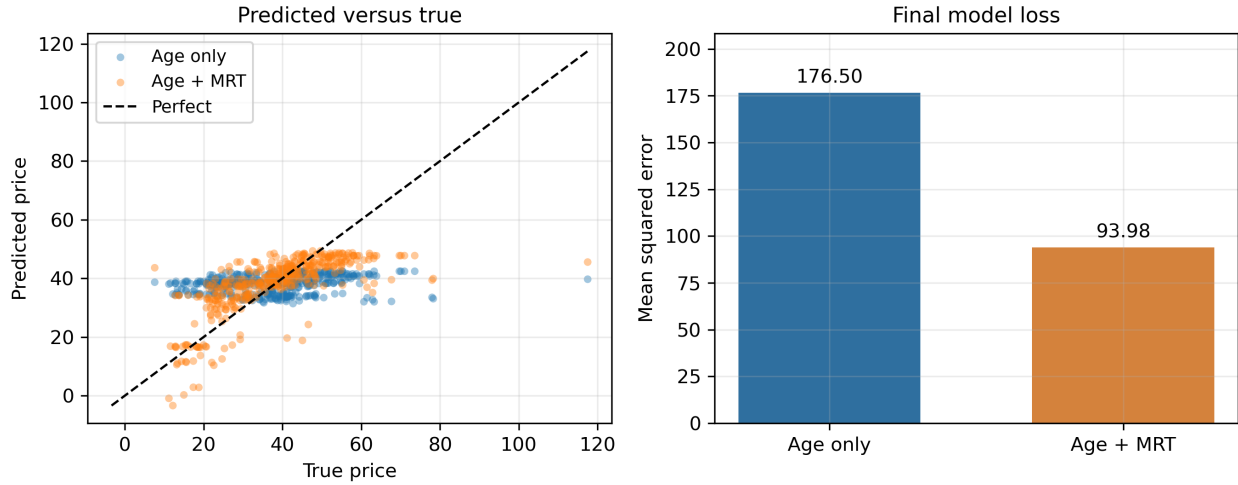


Figure 4: Comparison of predicted-versus-true price and final MSE.

The feature plots in Fig. 5 support the numerical result. House age contains some information, but the relationship is weak and noisy. MRT distance carries substantially more information: prices generally decline as distance increases. Feature scale was the main optimization limitation on the raw data. After normalization, the learning rate was stable and the loss plateaued well before 20,000 epochs, so the epoch count was not limiting. The extreme target near 117.5 and the restriction to a linear relationship remain clear limitations.

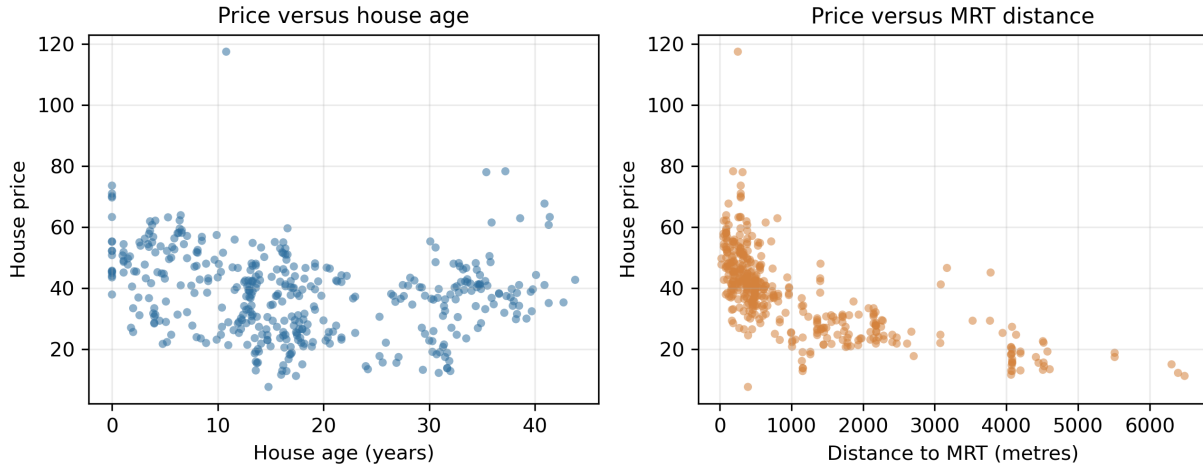


Figure 5: Price has a weak relationship with house age and a clearer negative relationship with MRT distance.

In a more careful version, I would investigate unusual observations, add more features, and test nonlinear terms. I would also reserve held-out data for evaluation, since the losses reported here measure fit to the same observations used for training and do not establish performance on new unseen houses.

5 Bonus

Setting the gradient equal to zero gives the normal equation

$$\mathbf{w}_{\text{closed}} = (X^T X)^{-1} X^T \mathbf{y}. \quad (11)$$

I solved $(X^T X)\mathbf{w} = X^T \mathbf{y}$ using `numpy.linalg.solve`, which is mathematically equivalent but avoids explicitly forming the inverse. The resulting weights were $[37.9802, -2.6288, -9.0871]^T$ with MSE 93.9798. After 20,000 epochs, the largest absolute difference between gradient-descent and closed-form weights was 1.78×10^{-12} , and the loss difference was 1.42×10^{-14} .

I defined “close enough” as every weight being within 0.001 of its closed-form value. With normalized features, this discrepancy has a negligible effect on a typical prediction relative to the model’s residual errors. Gradient descent first met this criterion after 5,268 epochs. The agreement confirms that the implemented gradient converges to the least-squares solution.

6 AI use

I used ChatGPT to help me with some of the coding commands. In particular, to refresh my memory on `pandas` and `matplotlib`. I also used it to do a final organisation of my notebook and the overleaf document.

References

- [1] I.-C. Yeh, “Real Estate Valuation,” UCI Machine Learning Repository, 2018. doi: [10.24432/C5J30W](https://doi.org/10.24432/C5J30W).